Deliverable 1 Report

Software Design & Development – Lost and Found System

1. Introduction

The Lost and Found System is designed to manage lost and found items in an organization. The system allows users to report lost items, record found items, and claim items.

To improve the structure of the system, several **creational design patterns** were applied. These patterns help organize how objects are created and make the system easier to maintain.

The patterns implemented in this project are:

- Singleton Pattern
- Factory Method Pattern
- Abstract Factory Pattern

2. Singleton Pattern – DBConnection

Problem

Multiple classes need to access the database. Creating many database connections could cause performance problems.

Solution

The **Singleton pattern** ensures that only **one DBConnection instance** exists in the system.

Implementation

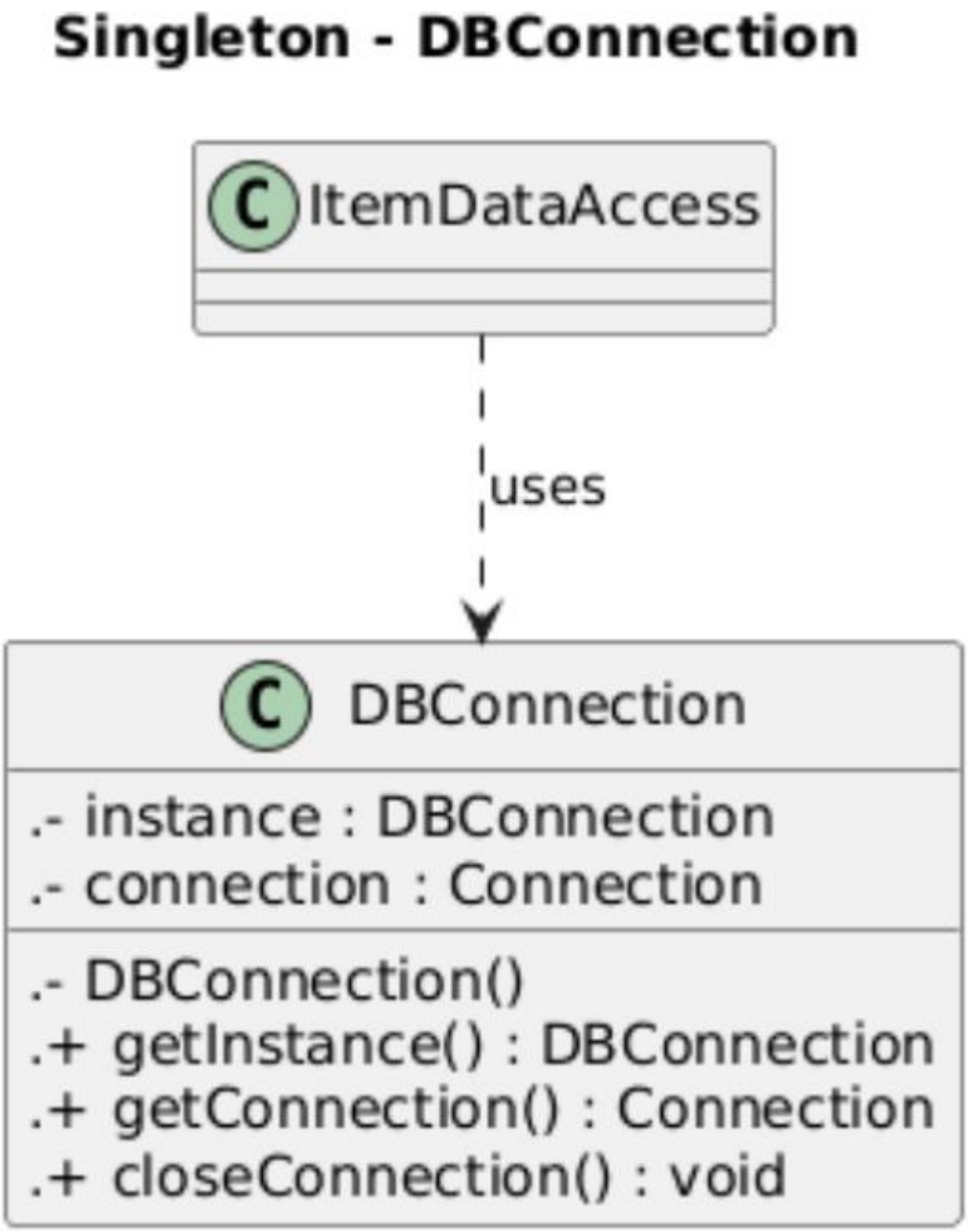
Class used:

dataaccess/DBConnection.java

DAO classes use the connection like this:

DBConnection.getInstance().getConnection()

UML Diagram


Singleton - DBConnection
ItemDataAccess
uses
DBConnection
.- instance : DBConnection
.- connection : Connection
.- DBConnection()
.+ getInstance() : DBConnection
.+ getConnection() : Connection
.+ closeConnection() : void

3. Singleton Pattern – SessionManager

Problem

The system must keep information about the currently logged-in user.

Solution

A **SessionManager Singleton** was created to store the current user information.

Implementation

Class used:

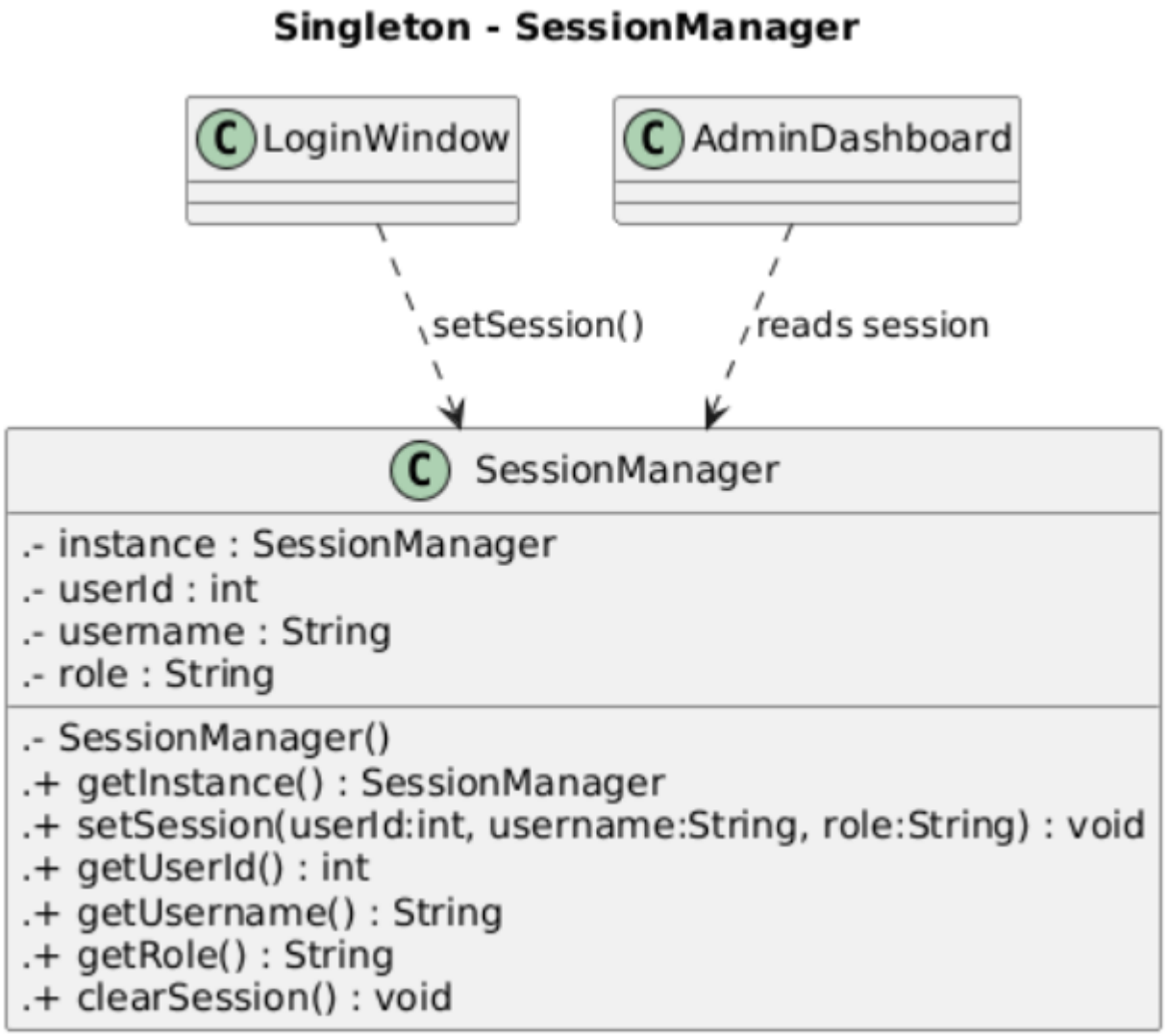
core/SessionManager.java

After login:

```
SessionManager.getInstance().setSession(userId, username, role);
```

Other classes can access the session information when needed.

UML Diagram


Singleton - SessionManager
LoginWindow
AdminDashboard
setSession()
reads session
SessionManager
.- instance : SessionManager
.- userId : int
.- username : String
.- role : String
.- SessionManager()
.+ getInstance() : SessionManager
.+ setSession(userId:int, username:String, role:String) : void
.+ getUserId() : int
.+ getUsername() : String
.+ getRole() : String
.+ clearSession() : void

5. Pattern Case 3 – Factory Method Pattern (Dashboard Creation)

Problem

The LoginWindow originally created dashboard windows directly using conditional statements such as:

- new AdminDashboard()
- new OfficerDashboard()
- new SearchWindow()

This approach tightly couples the LoginWindow with specific dashboard implementations.

Proposed Solution

The **Factory Method pattern** was implemented to delegate the responsibility of creating dashboard windows to separate creator classes.

This allows LoginWindow to work with a common interface rather than directly creating specific dashboard objects.

Implementation Summary

The Factory Method pattern is implemented using the following classes:

Abstract Creator:

factory/DashboardCreator.java

Concrete Creators:

factory/AdminDashboardCreator.java
factory/OfficerDashboardCreator.java
factory/StudentDashboardCreator.java

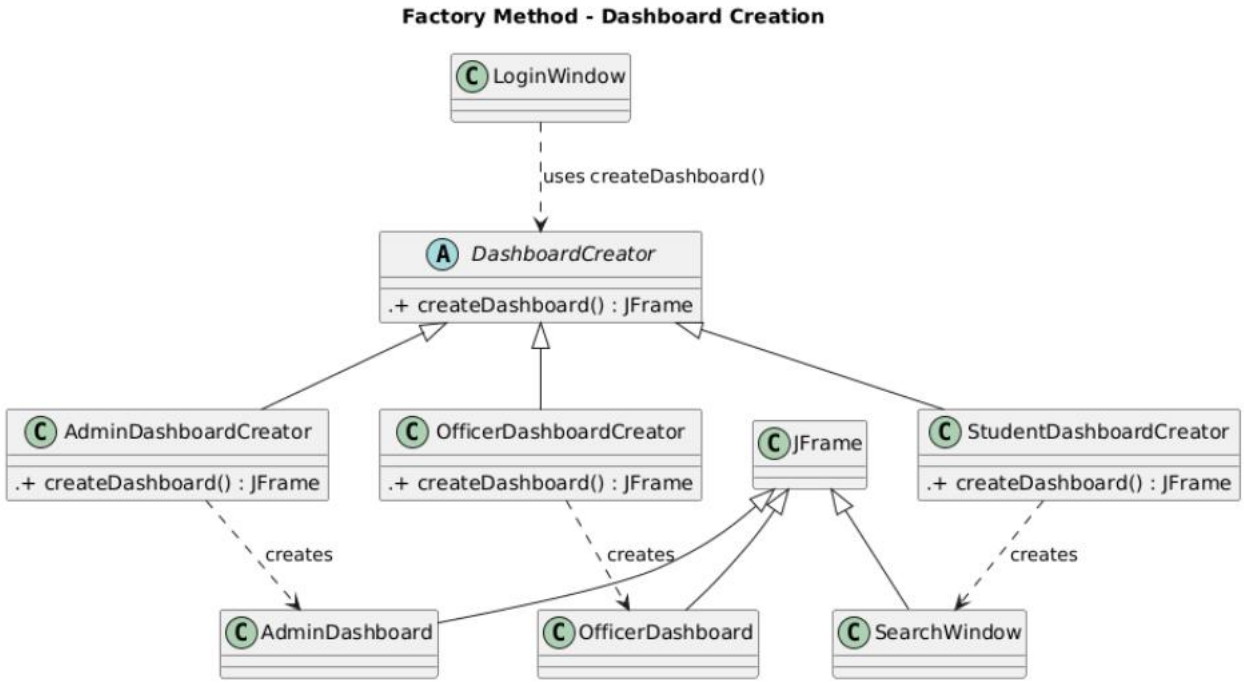
Each concrete creator implements the createDashboard() method and returns the appropriate dashboard window.

LoginWindow selects the correct creator based on the user role and calls the factory method to create the dashboard.

Result

The dashboard creation logic is separated from LoginWindow, making the system easier to extend if new user roles or dashboards are added in the future.

UML Diagram



6. Pattern Case 4 – Abstract Factory Pattern (DAO Family)

Problem

Controllers and other classes previously created DAO objects directly using statements such as:

```
new ItemDataAccess()
new UserDataAccess()
new ClaimDataAccess()
```

This tightly couples the system to specific DAO implementations.

Proposed Solution

The **Abstract Factory pattern** was introduced to create a family of DAO objects through a centralized factory interface.

This allows the system to easily switch between different DAO implementations if needed.

Implementation Summary

The abstract factory interface is defined in:

```
factory/dao/DataAccessFactory.java
```

Concrete factory implementations include:

```
factory/dao/SqlDataAccessFactory.java
factory/dao/MockDataAccessFactory.java
```

The SQL factory creates the actual DAO objects used in the system:

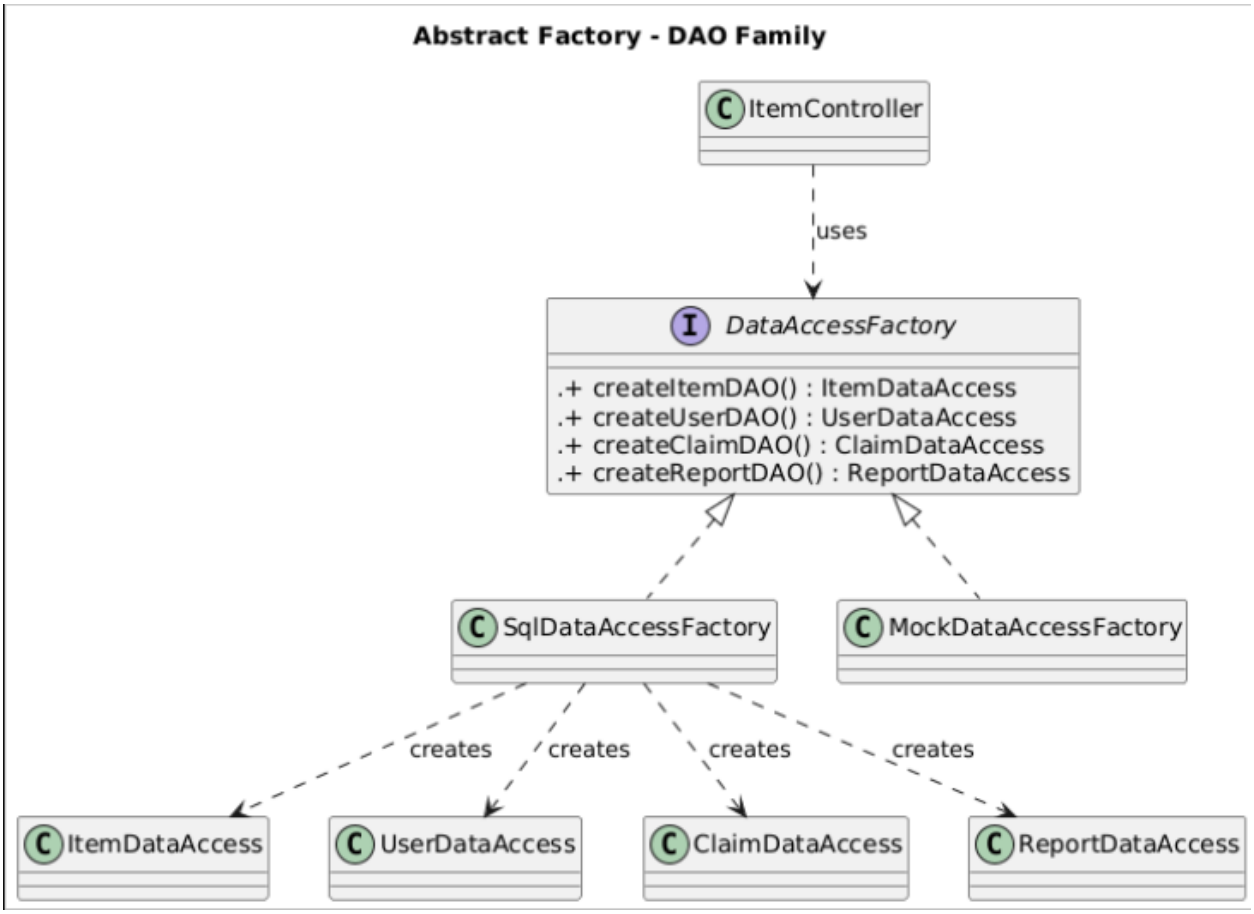
- ItemDataAccess
- UserDataAccess
- ClaimDataAccess
- ReportDataAccess

Controllers obtain DAO objects through the factory rather than directly instantiating them.

Result

The system becomes more flexible and decoupled from specific DAO implementations.

UML Diagram


Abstract Factory - DAO Family
ItemController
uses
DataAccessFactory
.+ createItemDAO() : ItemDataAccess
.+ createUserDAO() : UserDataAccess
.+ createClaimDAO() : ClaimDataAccess
.+ createReportDAO() : ReportDataAccess
SqlDataAccessFactory
MockDataAccessFactory
creates
creates
creates
creates
ItemDataAccess
UserDataAccess
ClaimDataAccess
ReportDataAccess

7. Pattern Case 5 – Abstract Factory Pattern (UI Component Family)

Problem

User interface components such as buttons, labels, and text fields were created directly within the UI classes. This makes it difficult to maintain consistent styles across different UI themes.

Proposed Solution

The **Abstract Factory pattern** was implemented to create families of UI components with consistent styling.

Two factories were created:

- LightUIFactory
- DarkUIFactory

Each factory produces a consistent set of UI components.

Implementation Summary

The abstract factory interface is defined in:

factory/ui/UIFactory.java

Concrete implementations include:

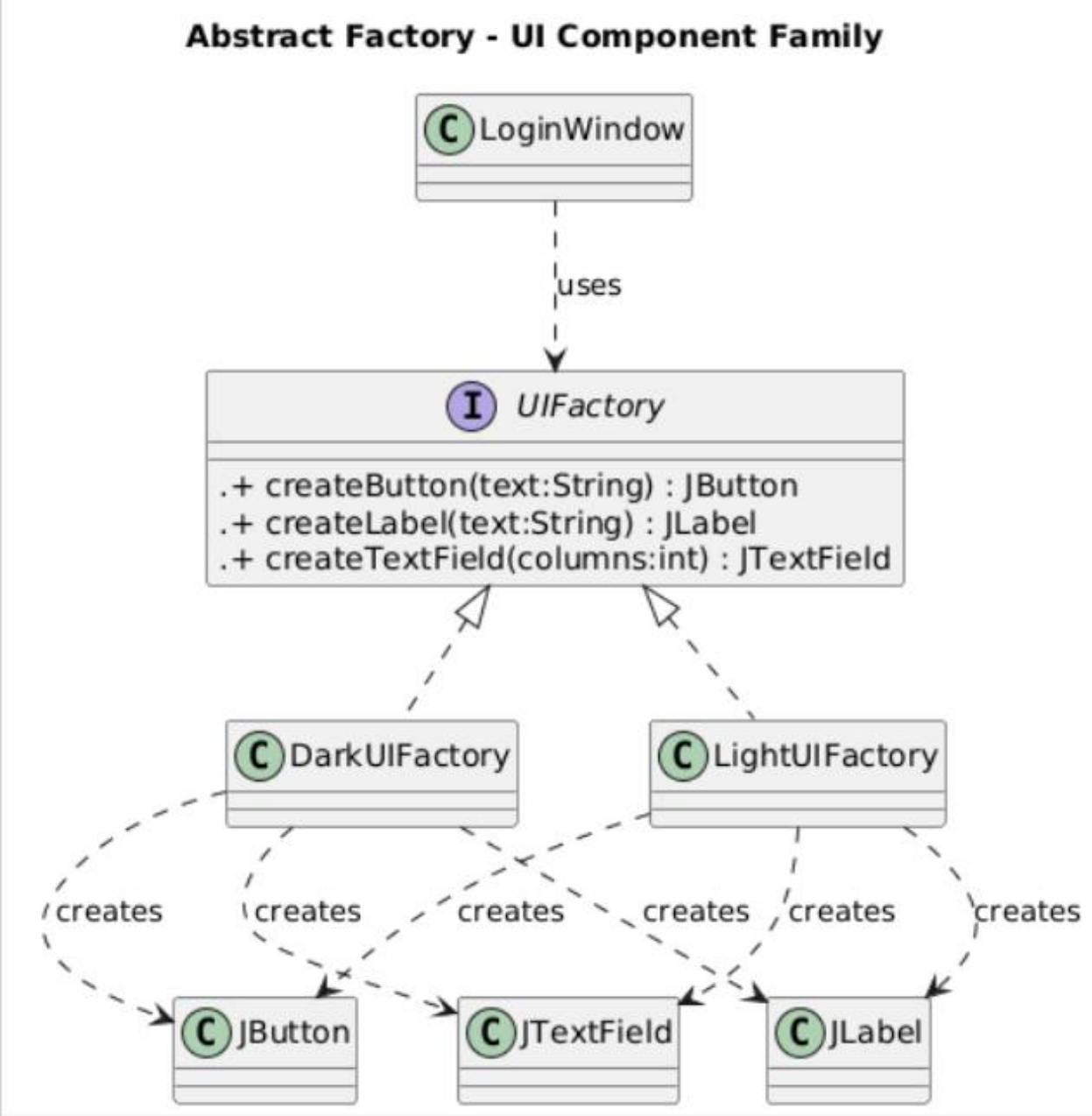
factory/ui/LightUIFactory.java
factory/ui/DarkUIFactory.java

UI classes such as LoginWindow use the UIFactory to create components instead of directly instantiating Swing components.

Result

The UI creation process becomes more flexible, allowing different visual themes to be applied without modifying the main UI logic.

UML Diagram


Abstract Factory - UI Component Family
LoginWindow
uses
UIFactory
.+ createButton(text:String) : JButton
.+ createLabel(text:String) : JLabel
.+ createTextField(columns:int) : JTextField
DarkUIFactory
LightUIFactory
creates
creates
creates
creates
creates
creates
JButton
JTextField
JLabel

8. Conclusion

In this deliverable, several creational design patterns were successfully integrated into the Lost and Found System to improve its architecture and maintainability.

The Singleton pattern ensures that shared resources such as database connections and user sessions are managed consistently. The Factory Method pattern decouples object creation from the login process, making the system more extensible. The Abstract Factory pattern centralizes the creation of related object families, improving flexibility and modularity.

These design patterns enhance the system's scalability and make future development and maintenance significantly easier.

---

9. Assumptions

- The database structure defined in the previous semester remains unchanged.
- The system operates with a single active user session at a time.
- DAO implementations currently connect to a SQL Server database.
- UI component factories demonstrate the concept of theme-based UI creation